CSS :has() Pseudo-class

Complete Guide: From Basics to Advanced

CSS Guide

May 10, 2026

Contents

Chapter 1

Introduction

1.1 What is the :has() Pseudo-class?

The `:has()` pseudo-class is a **parent selector** that allows you to style an element based on its children or subsequent siblings. It's one of the most revolutionary CSS features in recent years, finally solving the problem of styling parent elements based on their content.

```
/* Basic syntax: Style parent if it contains a child */
.parent:has(.child) {
    /* Styles applied to parent */
}

/* Practical example */
.card:has(img) {
    border-radius: 8px;
}
```

Before `:has()`, styling parents based on children was impossible with CSS alone. This pseudo-class revolutionizes responsive design and component-based styling.

1.2 Why is :has() Revolutionary?

1.2.1 The Problem Before :has()

Before this pseudo-class, there was no CSS-only way to:

- Style a parent element based on its children
- Change layout when certain elements are present
- Style a form based on input validation
- Create responsive components without JavaScript

Developers had to use:

- JavaScript to add classes
- CSS-in-JS solutions

- Complex HTML structures with fallback elements
- Server-side rendering workarounds

1.2.2 The Solution With :has()

```
/* Pure CSS: Style parent based on child content */
.form:has(input:invalid) {
    border-color: red;
}

.grid:has(> .item:nth-child(n+5)) {
    grid-template-columns: repeat(3, 1fr);
}

/* No JavaScript needed! */
```

1.3 Browser Support

Modern browsers now support `:has()`:

Browser	Version	Status
Chrome/Edge	v105+	**Full Support**
Firefox	v121+	**Full Support**
Safari	v15.4+	**Full Support**
Opera	v91+	**Full Support**
Internet Explorer	N/A	Not Supported

Chapter 2

Core Concepts

2.1 Understanding :has() Syntax

2.1.1 Basic Syntax

```
/* Element that HAS a direct child matching selector */
element:has(> selector) {
    /* Styles */
}

/* Element that HAS any descendant matching selector */
element:has(selector) {
    /* Styles */
}

/* Element that HAS a sibling matching selector */
element:has(~ selector) {
    /* Styles */
}
```

2.1.2 Breaking Down the Syntax

- **:has()** - The pseudo-class name
- **>** - Direct child combinator
- **(space)** - Descendant combinator
- **~** - General sibling combinator
- **selector** - Any CSS selector inside parentheses

2.2 Types of :has() Selectors

2.2.1 Direct Child Selector

```
/* Style parent only if it has a direct child matching selector */
.container:has(> .featured) {
    background: #f0f0f0;
}

/* Example: Card with featured badge */
.card:has(> .badge-featured) {
    border: 2px solid gold;
    box-shadow: 0 0 10px rgba(255, 215, 0, 0.5);
}
```

2.2.2 Descendant Selector

```
/* Style parent if it contains any descendant matching selector */
.form:has(input) {
    padding: 20px;
}

/* Example: Section with any level of error */
.section:has(.error) {
    background: #ffe6e6;
}

/* Can be nested multiple levels deep */
div:has(section article p) {
    /* Styles if div contains article > p deep inside */
}
```

2.2.3 Sibling Selector

```
/* Style element if a following sibling matches selector */
.label:has(~ input:checked) {
    font-weight: bold;
    color: #667eea;
}

/* Example: Highlight when checkbox is checked */
.checkbox-label:has(~ input[type="checkbox"]:checked) {
    background: #e8f4f8;
}
```

Chapter 3

Practical Examples

3.1 Example 1: Form Validation Styling

```
/* Base form styling */
.form-group {
    margin-bottom: 20px;
}

label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

input {
    width: 100%;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
}

/* Style group when input is invalid */
.form-group:has(input:invalid) {
    background: #fff5f5;
    padding: 10px;
    border-radius: 4px;
    border-left: 4px solid #e74c3c;
}

.form-group:has(input:invalid) input {
    border-color: #e74c3c;
    background: #ffe6e6;
}

/* Style group when input is valid */
.form-group:has(input:valid) {
    border-left: 4px solid #27ae60;
}
```

```
.form-group:has(input:valid) input {
    border-color: #27ae60;
}

/* Style label based on input state */
.form-group:has(input:invalid) label {
    color: #e74c3c;
}

.form-group:has(input:valid) label {
    color: #27ae60;
}
```

3.2 Example 2: Card Component Variations

```
/* Base card styling */
.card {
    background: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
    transition: all 0.3s ease;
}

/* Card with image */
.card:has(img) {
    border-radius: 12px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}

.card:has(img) img {
    width: 100%;
    height: 200px;
    object-fit: cover;
}

/* Card with featured badge */
.card:has(.badge-featured) {
    border: 2px solid #f39c12;
    background: linear-gradient(135deg,
                                #fff9e6 0%,
                                #ffffff 100%);
}

.card:has(.badge-featured) .content {
    padding: 20px;
}

/* Card with action buttons */
.card:has(.card-actions) {
    display: flex;
    flex-direction: column;
```

```
}

.card:has(.card-actions) .content {
    flex: 1;
}

/* Card with multiple items */
.card:has(> .item:nth-child(n+3)) {
    padding: 20px;
}

/* Card with hover effect on image */
.card:has(img):hover img {
    transform: scale(1.05);
}
```

3.3 Example 3: Responsive Grid Layout

```
/* Grid container */
.grid {
    display: grid;
    gap: 20px;
    grid-template-columns: repeat(2, 1fr);
}

/* Few items: larger cards */
.grid:has(> .item:nth-child(n+5)) {
    grid-template-columns: repeat(3, 1fr);
}

/* Many items: more columns */
.grid:has(> .item:nth-child(n+9)) {
    grid-template-columns: repeat(4, 1fr);
}

/* Just one item: full width */
.grid:has(> .item:nth-child(1):last-child) {
    grid-template-columns: 1fr;
}

/* Item styling */
.item {
    background: white;
    padding: 20px;
    border-radius: 8px;
}

/* Item with image: larger */
.grid:has(> .item:nth-child(n+3)) .item:has(img) {
    grid-column: span 2;
    grid-row: span 2;
}
```

3.4 Example 4: Navigation Menu States

```
/* Navigation styling */
nav ul {
    list-style: none;
    display: flex;
    gap: 0;
}

nav li {
    position: relative;
}

nav a {
    display: block;
    padding: 15px 20px;
    color: white;
    text-decoration: none;
    background: #2c3e50;
}

/* Submenu styling */
nav ul ul {
    position: absolute;
    top: 100%;
    left: 0;
    flex-direction: column;
    background: #1a252f;
    min-width: 200px;
    opacity: 0;
    visibility: hidden;
    transition: all 0.3s ease;
}

/* Show submenu when hovering parent */
nav li:has(> ul):hover > ul {
    opacity: 1;
    visibility: visible;
}

/* Style parent with submenu */
nav li:has(> ul) > a::after {
    content: ' ';
    font-size: 12px;
}

/* Active menu item */
nav a:is(.active, :hover) {
    background: #667eea;
}

/* Menu item with active submenu */
nav li:has(a.active) > a {
```

```
    background: #667eea;
}
```

3.5 Example 5: Checkbox Toggle Groups

```
/* Checkbox wrapper */
.checkbox-wrapper {
    display: flex;
    align-items: center;
    padding: 10px;
    border-radius: 4px;
    transition: all 0.2s ease;
}

/* Hide actual checkbox */
.checkbox-wrapper input[type="checkbox"] {
    display: none;
}

/* Checkbox label styling */
.checkbox-wrapper label {
    margin: 0;
    cursor: pointer;
    user-select: none;
}

/* Style wrapper when checked */
.checkbox-wrapper:has(input[type="checkbox"]:checked) {
    background: #e8f4f8;
    border-left: 4px solid #3498db;
}

.checkbox-wrapper:has(input[type="checkbox"]:checked) label {
    font-weight: bold;
    color: #3498db;
}

/* Custom checkbox styling */
.checkbox-wrapper::before {
    content: '';
    margin-right: 10px;
    font-size: 18px;
    color: #999;
}

.checkbox-wrapper:has(input[type="checkbox"]:checked)::before {
    content: '';
    color: #3498db;
}

/* Radio button variant */
.radio-wrapper {
```

```
    display: flex;
    gap: 10px;
}

.radio-group:has(input[type="radio"]:checked) {
    background: #f0f0f0;
    padding: 20px;
    border-radius: 8px;
}
```

Chapter 4

Advanced Techniques

4.1 Complex Selectors with :has()

4.1.1 Combining Multiple Conditions

```
/* Element with multiple required children */
.component:has(> .header):has(> .body):has(> .footer) {
    display: flex;
    flex-direction: column;
    min-height: 400px;
}

/* Element with specific child configuration */
.list:has(> li:nth-child(1)) {
    /* Has at least one item */
    padding: 10px;
}

/* Multiple conditions with OR (comma) */
.element:has(a),
.element:has(button) {
    cursor: pointer;
}
```

4.1.2 Combining with Pseudo-classes

```
/* Element with active child */
.nav:has(.item:is(.active, :hover)) {
    background: #f5f5f5;
}

/* Element with checked input */
.form:has(input:checked) {
    border: 2px solid #27ae60;
}

/* Element with empty state */
.container:has(> :empty) {
```

```
    display: none;
}

/* Element with focused child */
.form-group:has(input:focus) {
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
    border-color: #667eea;
}
```

4.2 Performance Considerations

4.2.1 Efficient :has() Usage

```
/* Good: Specific selector */
.card:has(> img) {
    /* Efficient: Direct child only */
}

/* Less efficient: Broad descendant */
.card:has(img) {
    /* Searches all descendants */
}

/* Avoid: Too many :has() in cascade */
.container:has(.item):has(.item:hover):has(.item.active) {
    /* Multiple :has() can impact performance */
}

/* Better: Use more specific selectors */
.container:has(.item.active) {
    /* More direct targeting */
}
```

4.3 Browser Compatibility Fallback

```
/* Default styles (no :has() support) */
.form-group {
    border: 1px solid #ddd;
}

/* Enhanced with :has() support */
@supports selector(:has(> *)) {
    .form-group:has(input:invalid) {
        border-color: #e74c3c;
        background: #fff5f5;
    }

    .form-group:has(input:valid) {
        border-color: #27ae60;
    }
```

```
}
```

Chapter 5

Real-World Use Cases

5.1 Use Case 1: Product Card System

```
.product-card {
    background: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

/* Card with sale badge: highlight it */
.product-card:has(.badge-sale) {
    border: 2px solid #e74c3c;
    background: linear-gradient(135deg,
                                #ffe6e6 0%,
                                #ffffff 100%);
}

/* Card with image: add hover effect */
.product-card:has(img) {
    cursor: pointer;
}

.product-card:has(img):hover {
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);
    transform: translateY(-5px);
}

/* Card with low stock warning */
.product-card:has(.stock-warning) {
    opacity: 0.8;
    border-left: 4px solid #f39c12;
}

/* Card with rating: show prominently */
.product-card:has(.rating-5) {
    background: linear-gradient(135deg,
                                #fff9e6 0%,
                                #ffffff 100%);
```

```
}
```

5.2 Use Case 2: Form with Complex Validation

```
.form {
    max-width: 600px;
}

.field {
    margin-bottom: 20px;
}

/* Field with error: show error styling */
.field:has(.error-message) {
    background: #fff5f5;
    padding: 15px;
    border-radius: 4px;
    border-left: 4px solid #e74c3c;
}

.field:has(.error-message) label {
    color: #e74c3c;
    font-weight: bold;
}

.field:has(.error-message) input,
.field:has(.error-message) textarea {
    border-color: #e74c3c;
    background: #ffe6e6;
}

/* Field with success state */
.field:has(.success-message) {
    background: #f0fdf4;
    border-left: 4px solid #27ae60;
}

.field:has(.success-message) input {
    border-color: #27ae60;
}

/* Field with help text */
.field:has(.help-text) {
    margin-bottom: 30px;
}

.field:has(.help-text) .help-text {
    font-size: 12px;
    color: #666;
    margin-top: 5px;
}
```

5.3 Use Case 3: Dynamic Layout Selection

```
.gallery {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}

/* Change layout based on item count */
.gallery:has(> .item:nth-child(1):nth-last-child(1)) {
    /* Single item */
    grid-template-columns: 1fr;
}

.gallery:has(> .item:nth-child(1):nth-last-child(2)),
.gallery:has(> .item:nth-child(2):nth-last-child(1)) {
    /* Two items */
    grid-template-columns: repeat(2, 1fr);
}

.gallery:has(> .item:nth-child(1):nth-last-child(4)),
.gallery:has(> .item:nth-child(4):nth-last-child(1)) {
    /* Four items or more */
    grid-template-columns: repeat(2, 1fr);
}

.gallery:has(> .item:nth-child(1):nth-last-child(n+5)) {
    /* Five or more items */
    grid-template-columns: repeat(4, 1fr);
}
```

Chapter 6

Browser Support & Fallbacks

6.1 Current Support

Browser	Version	Release	Status
Chrome	105+	Sep 2022	**Full Support**
Edge	105+	Oct 2022	**Full Support**
Safari	15.4+	Mar 2022	**Full Support**
Firefox	121+	Dec 2023	**Full Support**
Opera	91+	Oct 2022	**Full Support**
IE 11	N/A	N/A	Not Supported

6.2 Progressive Enhancement

```
/* Fallback: Works without :has() */
.card {
    border: 1px solid #ddd;
}

/* Enhanced: Only in modern browsers */
@supports selector(:has(> *)) {
    .card:has(.featured-badge) {
        border: 2px solid gold;
        background: #fffacd;
    }
}
```

Chapter 7

Best Practices

7.1 Key Recommendations

7.1.1 1. Use :has() for Component Styling

```
/* Good: Component adjusts based on content */
.card:has(video) {
    aspect-ratio: 16 / 9;
}

.card:has(img) {
    aspect-ratio: 1;
}
```

7.1.2 2. Be Specific with Child Selectors

```
/* Good: Direct child only */
.list:has(> li.highlight) {
    background: #f5f5f5;
}

/* Less efficient: All descendants */
.list:has(.highlight) {
    background: #f5f5f5;
}
```

7.1.3 3. Combine with @supports for Safety

```
@supports selector(:has(> *)) {
    .element:has(.child) {
        /* Modern browser styles */
    }
}
```

7.1.4 4. Test Cross-Browser

Always test :has() functionality in:

- Chrome 105+
- Firefox 121+
- Safari 15.4+
- Edge 105+

7.2 Common Patterns

Pattern	Use Case
`:has(> child)`	Direct child styling
`:has(selector:state)`	State-based styling
`:has(  sibling)`	Sibling relationships
`:has(nth-child(n))`	Count-based styling
`:has(:is())`	Multiple conditions

Chapter 8

Common Issues & Solutions

8.1 Issue 1: :has() Not Working

Problem: Styles not applying with :has().

Causes:

- Browser too old (pre-2022)
- Incorrect selector syntax
- Targeting wrong element level

Solution:

```
/* Check browser support */
@supports selector(:has(> *)) {
    /* Safe to use :has() */
    .element:has(.child) {
        color: red;
    }
}
```

8.2 Issue 2: Performance Concerns

Problem: Page feels slow with :has().

Solution:

```
/* Avoid overly complex selectors */
.element:has(div span p a) {
    /* Too deep! */
}

/* Use more direct selectors */
.element:has(> .target) {
    /* More efficient */
}

/* Limit repetition */
/* Avoid many :has() in sequence */
```

8.3 Issue 3: Specificity Problems

Problem: Styles being overridden unexpectedly.

Solution:

```
/* :has() has specificity of the selector inside */
.card:has(.badge) {
    /* Specificity: element + class + class = 2 */
}

/* More specific if needed */
.card.featured:has(> .badge) {
    /* Specificity: element + class + class + class = 3 */
}
```

Chapter 9

Summary & Quick Reference

9.1 When to Use :has()

Use `:has()` when:

- Styling components based on children
- Creating responsive component layouts
- Styling form elements based on validation
- Building interactive UI without JavaScript
- Checking for element presence

9.2 When NOT to Use

- Need IE 11 support
- Supporting browsers older than 2022
- Complex performance-critical applications
- Can accomplish with simpler selectors

9.3 Quick Reference

9.3.1 Common :has() Patterns

```
/* Has direct child */
.parent:has(> .child) { }

/* Has any descendant */
.parent:has(.descendant) { }

/* Has sibling after element */
.element:has(~ .sibling) { }
```

```
/* Has element in specific state */
.form:has(input:invalid) { }

/* Has nth child */
.list:has(> li:nth-child(n+5)) { }

/* Has multiple conditions */
.card:has(.image):has(.title) { }

/* Has element matching selector */
.menu:has(.item:is(.active, :hover)) { }
```

9.3.2 Complete Practical Template

```
/* Base component styling */
.component {
    background: white;
    border-radius: 8px;
    padding: 20px;
}

/* Component variations based on content */
@supports selector(:has(> *)) {
    /* With image */
    .component:has(img) {
        padding: 0;
        overflow: hidden;
    }

    .component:has(img) img {
        width: 100%;
        height: 200px;
        object-fit: cover;
    }

    /* With error */
    .component:has(.error) {
        border: 2px solid #e74c3c;
        background: #fff5f5;
    }

    /* With success */
    .component:has(.success) {
        border: 2px solid #27ae60;
        background: #f0fdf4;
    }

    /* With multiple children */
    .component:has(> .item:nth-child(n+3)) {
        display: grid;
        grid-template-columns: repeat(2, 1fr);
        gap: 15px;
    }
```

```
}
```

Thank you for reading!

For more information, visit MDN Web Docs and web.dev